

FE Developer

김민찬

사용자 행동을 제품 경험으로 연결하는 프론트엔드 개발자

Email ydjm1994@gmail.com

Phone 010-9957-3068

Blog velog.io/@kimjih94/posts

SUMMARY

사용자 행동과 데이터 흐름을 바탕으로 복잡한 도메인을 이해하기 쉬운 경험으로 구현해온 프론트엔드 개발자입니다.

Agent UI, 대용량 데이터 시각화, 도구형 UI 성능 개선 경험을 바탕으로 콘텐츠 탐색과 인터랙션 흐름을 개선하는데 기여할 수 있습니다.

핵심역량

사용자 흐름 설계 및 개선

렌더링 성능 최적화

Agent UI 구현

재사용 가능한 UI 설계

협업 기반 문제정의

EDUCATION

경북대학교

전자공학부

SSAFY

웹개발

TOOL

DEV

React · TypeScript · TailwindCSS
TanStack Query · Zustand

AI · AGENT

Claude AI · Cursor AI

COLLABORATION

Figma · Notion · ClickUp · Slack

EXPERIENCE

(주) 아하랩스

2024.11 — 재직중

제조 AI MLOps 플랫폼 / CNC 관제 대시보드

- Agent UI 공통화 및 재사용 구조 설계
서로 다른 Agent 프로젝트 3개에서 반복되던 입력 폼·실행 상태·결과 표시 UI를 공통화 재사용 가능한 구조로 설계해 약 60% 수준의 코드 재사용률을 달성
- 대용량 데이터 시각화 성능 개선
5천만 건 이상 시계열 데이터를 다루는 분석 화면에서 고빈도 인터랙션 병목을 분석 다운샘플링, 상태 분리, 커스텀 렌더링을 적용해 UX 흐름이 끊기지 않도록 개선
- 도구형 UI 사용 흐름 설계
시계열 데이터, 워크플로우 등 여러 단계를 거쳐 조작하는 화면의 UX 흐름을 시각화 기획·디자인·백엔드와 구현 가능한 단위로 요구사항을 조율하며 제품 고도화

티맥스 클라우드

2023.08 — 2024.11

슈퍼앱 내부 Code Editor / 클라우드 플랫폼 관리 콘솔

- GAIA PX 프론트엔드 단독 개발
Monaco Editor 기반 웹 에디터에서 자동완성, 파일트리, 에러마킹 등 핵심 기능을 구현 저장·검색 성능 저하를 개선하기 위해 IndexedDB 기반 로컬 저장 구조를 도입
- TCP IaaS 네트워크 설정 UI 개발
Kubernetes 기반 컨테이너 오케스트레이션 환경에서 보안 그룹, 로드밸런서 등 네트워크 설정을 사용자가 확인하고 조작할 수 있는 화면으로 구현
- 공용 컴포넌트 설계 및 협업
반복되는 설정 UI와 상태 표시 요소를 공용 컴포넌트로 추출해 코드 중복을 줄이고, 다른 팀에서도 재사용 가능한 형태로 설계

(주) 아하랩스

2024.11 — 재직중

제조 AI MLOps 플랫폼 (DAISY) / CNC 관제 대시보드

01 Agent UI 공통화 및 재사용 구조 설계

- 고객사별 Agent 웹앱 요구가 늘어나며 입력 폼, 실행 상태, 결과 표시 등 반복 UI와 로직을 프로젝트마다 중복 구현해야 하는 문제가 발생
- 프로젝트별 요구사항은 분리하되, 반복되는 인터랙션과 상태 처리 로직은 공통화해야 대응 속도와 유지보수성을 함께 높일 수 있다고 판단
- 공통 컴포넌트와 Hook을 분리해 별도 패키지로 배포했고, 약 60% 수준의 코드 재사용률 달성
- 공통화는 반복되는 요구사항과 달라질 수밖에 없는 요구사항을 구분하는 설계 작업이라는 점을 배움

02 대용량 시계열 데이터 시각화 성능 개선

- 대량 시계열 데이터(5천만건)에서는 차트 라이브러리가 데이터를 임의로 생략하거나 다운샘플링하는 문제 발생
- 단순 렌더링 최적화만으로는 한계가 있어, 상태 구독 범위와 데이터 표시 방식을 함께 재설계해야 사용자의 탐색 흐름을 유지할 수 있다고 판단
- 자체적으로 다운샘플링 알고리즘(LTTB)을 적용해 대량 데이터 환경에서도 60FPS 수준의 인터랙션을 목표로 개선
- 논문을 통해 다양한 다운샘플링 알고리즘과 라이브러리에서 사용하던 Zustand에 대한 이해도를 획득

03 복잡한 도구형 UI의 사용 흐름 설계

- 워크플로우처럼 여러 단계를 순서대로 수행해야 하는 화면에서 QA와 신규 개발자가 초기 사용 흐름을 이해하기 어려운 문제 발생
- 기능을 단순히 배치하는 것만으로는 부족하며, 사용자가 현재 단계와 다음 행동을 화면 안에서 자연스럽게 이해할 수 있어야 한다고 판단
- 기획팀과 협업해 상태별 색상, 단계별 UI, 결과 표시 방식을 정리하고 작업 순서에 맞게 화면 흐름을 개선해 초기 사용자 UX 개선
- 복잡한 도구형 UI에서는 기능의 개수보다 사용자가 작업 맥락을 잃지 않고 다음 단계로 이어갈 수 있는 흐름 설계가 더 중요하다는 점을 배움

티맥스 클라우드

2023.08 — 2024.10

슈퍼앱 내부 Code Editor / 클라우드 플랫폼 관리 콘솔

01 GAIA PX 프론트엔드 단독 개발

- 사내 SDK 개발과 리팩토링 작업을 웹 환경에서 수행해야 했지만, IDE의 고급기능을 지원하지 못했기에 DX 저하가 발생
- 브라우저 안에서도 개발자가 작업 맥락을 유지하려면 파일 구조, 자동완성, 오류 표시, 실행 흐름까지 함께 제공해야 한다고 판단
- Monaco Editor 기반 웹 에디터에서 핵심 기능 구현 및 저장·검색 성능 저하를 개선하기 위해 IndexedDB 기반 로컬 저장 구조를 도입
- DX는 단순히 편의 기능을 추가하는 것이 아니라, 개발자가 반복적으로 지拂하는 인력 및 시간적 비용을 줄이는 방법을 학습

02 TCP IaaS 네트워크 설정 UI 개발

- Kubernetes 기반 컨테이너 오케스트레이션 환경에서 네트워크 정보를 GUI로 설정하고 싶다는 요구 발생
- 보안 그룹, 로드밸런서 등 네트워크 설정을 사용자가 확인하고 조작할 수 있는 화면으로 구현
- TanStack Query와 MobX를 활용해 서버 데이터와 화면 상태를 관리
- 전문성이 필요한 복잡한 도메인을 사용자가 친숙하게 사용할 수 있는 UI로 풀어내는 방법 학습

03 공용 컴포넌트 설계 및 협업

- 클라우드 설정 화면에서 유사한 입력 폼, 상태 표시, 설정 UI가 반복되며 코드 중복과 유지보수 부담이 커지는 문제 발생
- 반복되는 설정 UI와 상태 표시 요소는 개별 화면에 흩어두기보다 공용 컴포넌트로 분리해야 개발 속도와 유지보수성을 함께 높일 수 있다고 판단
- 공용 컴포넌트로 추출해 모노레포를 발행하여 다른 팀에서도 재사용 가능한 형태로 설계
- 공용 컴포넌트 설계는 반복되는 요구사항을 안정적으로 흡수할 수 있는 협업 기반을 만드는 일이라는 점을 학습

김민찬

사용자 행동을 제품 경험으로 연결하는 프론트엔드 개발자

01 오늘의집에 지원한 이유

오늘의집에 지원한 이유는 온라인 콘텐츠 경험이 실제 생활의 변화로 이어지는 제품이라는 점에 매력을 느꼈기 때문입니다.

최근 이사를 준비하고 생활 공간을 다시 구성하면서 오늘의집을 자주 사용했습니다. 콘텐츠를 보며 아이디어를 얻고, 상품을 비교하고, 실제 공간에 적용할 방법을 고민하는 과정에서 오늘의집이 단순한 커머스 서비스가 아니라 사용자의 생활 변화를 돕는 서비스라는 점을 체감했습니다. 또한 오늘의집 오프라인 매장을 방문하며, 온라인에서 시작된 인테리어 탐색 경험이 오프라인 경험으로 확장되는 방식에도 흥미를 느꼈습니다.

오늘의집 콘텐츠 개발팀에서도 사용자의 행동과 니즈를 바탕으로 콘텐츠를 더 잘 발견하고, 이해하고, 다음 행동으로 이어갈 수 있는 웹 경험을 만드는 데 기여하고 싶습니다.

02 AI Agent UI와 공통 UI 구조를 직접 설계하고 배포한 경험

아하랩스에서는 고객사별 Agent 웹앱 요구가 늘어나며 입력 폼, 실행 상태, 결과 표시 등 반복되는 UI와 로직을 프로젝트마다 중복 구현해야 하는 문제가 발생했습니다. 초기에는 각 프로젝트에 맞춰 구현했지만, 프로젝트가 세 개 이상으로 늘어나면서 같은 기능을 다시 만들거나 여러 레포지토리에 동일한 수정사항을 반영해야 하는 유지보수 비용이 커졌습니다.

저는 프로젝트별 요구사항은 분리하되, 반복되는 인터랙션과 상태 처리 로직은 공통화해야 한다고 판단했습니다. 이를 위해 공통 컴포넌트와 Hook을 분리하고, 신규 Agent 프로젝트에서 재사용할 수 있는 형태로 구조화했습니다. 이후 별도 패키지 형태로 배포해 새로운 Agent 프로젝트에서도 기존 구현을 복사하는 대신 공통 모듈을 기반으로 빠르게 화면을 구성할 수 있도록 했고, 프로젝트 간 약 60% 수준의 코드 재사용률을 달성했습니다.

이 경험은 오늘의집 콘텐츠 개발팀의 업무와도 맞닿아 있다고 생각합니다. 콘텐츠와 커뮤니티 화면은 계속 실험되고 변화하지만, 그 안에는 탐색, 저장, 피드백, 추천, 전환처럼 반복되는 사용자 흐름이 존재합니다. 저는 반복되는 UI 패턴과 달라질 수밖에 없는 요구사항을 구분해 확장 가능한 프론트엔드 구조로 만드는 데 기여할 수 있습니다.

03 프론트엔드의 경계를 넘어 필요한 기술을 학습하고 적용한 경험

백엔드의 부담이 커지는 환경에서는 GraphQL을 학습하며 클라이언트가 필요한 데이터를 더 명확히 선언하고, API 변경 비용을 줄일 수 있는 구조를 고민했습니다. 비록 현업 제품에 깊게 적용하지는 못했지만, 대규모 서비스들이 GraphQL을 선택하는 이유와 프론트엔드 개발자가 데이터 요구사항을 더 주도적으로 다룰 수 있는 방식을 이해하는 계기가 되었습니다.

또한 배포와 운영 과정에서 반복적으로 발생하던 수작업을 줄이기 위해 GitLab CI/CD를 학습하고 구성했습니다. MR 등록 시 테스트가 실행되고, 병합 이후 파이프라인을 통해 자동 배포되도록 구성해 프론트엔드 산출물이 수동으로 전달되거나 오래된 빌드가 배포될 수 있는 위험을 줄이고자 했습니다.

저는 프론트엔드 개발자가 화면만 구현하는 역할에 머물기보다, 데이터 흐름, 배포 과정, 도메인 구조를 함께 이해할 때 더 나은 제품을 만들 수 있다고 생각합니다.

이처럼 복잡한 기능을 사용자가 실행 가능한 흐름으로 바꾼 경험을 바탕으로, 코드잇에서도 학습자와 내부 사용자가 **더 쉽게 이해하고 안정적으로 사용할 수 있는 제품**을 만드는 데 기여하고 싶습니다.